RPA UI Interaction Patterns

1 Extraction Patterns

1.1 Pattern 1: Find element

Description: The Find Element pattern enables a bot to establish a reference to a target UI element. Within the AOMC model, Find Element represents an interaction action that connects a bot to an interaction object through context-dependent identification methods.
Motivation: Automation tasks rarely operate directly on applications as a standalone entity, instead, they interact with specific elements such as buttons, text fields, and images. The Find Element pattern works as a prerequisite step by separating element identification from subsequent actions, such as input and click. Successful bot operation relies on accurate object identification.
Pattern variants: Find Element manifests with three variants, each corresponding to a distinct interaction strategy based on the interface abstraction level, as summarised in Table 1.

Table 1. Variants of Find element pattern

Variant	Action	Object	Method	Context
1	Find	HTML element	DOM parsing	Web environment
2		HTML element, UI element	UI Automation tree parsing	Web environment, Desktop environment
3		HTML element, UI element, Visual element	Visual recognition	Web environment, Desktop environment, Screen environment

Applicability:

- The bot must be capable of executing JavaScript or other (e.g., IE COM) scripts.
- The bot must be capable of executing accessibility patterns to UI hierarchy (e.g., UI Automation Framework).
- The bot must be capable of conducting image recognition, this can be either by invoking an external service or built-in function.
- If the target object is an HTML element, the webpage must be fully loaded to ensure that the bot can accurately capture all elements that meet the criteria.

Operation: Bot accesses the DOM, UI hierarchy, or the image rendered on computer screen, based on the user-defined criteria. It identifies the elements that match the criteria. The bot can be configured to identify the required number of elements (the first matching instance, all matching instances, or a specific number of instances). If no element matches the specified criteria, appropriate error handling mechanism should be triggered.
Example: A bot automates data entry in a desktop accounting application. Before interacting with the "Submit" button, the bot applies selector criteria within the UI Automation hierarchy to locate the button element based on its accessible name and control type. Once identified, the resolved element reference is used by a subsequent interaction pattern to activate the button.
Tool Evaluation: See Table 2.

Table 2. Tool evaluation of Find Element

Tool	Version	Variant	Score	Justification
UiPath	Studio 2025.0.161	Find HTML element	+	Directly supported through "Find Element"
		Find UI element	+	Directly supported through "Find Element"
		Find visual element	+	Directly supported through "Find Element"
Automation Anywhere	Community Edition build 29082	Find HTML element	±	Integrated into "Recorder: Capture"
		Find UI element	±	Integrated into "Recorder: Capture"
		Find visual element	±	Directly supported through "Image Recognition: Find image in window"
Power Automate	2.53.180.25037	Find HTML element	+	Directly supported through "If web page contains"
		Find UI element	+	Directly supported through "If window contains"
		Find visual element	+	Directly supported through "If image"

1.2 Pattern 2: Read element

Description: This pattern enables a bot to read the attributes or properties of a target element. It operationalises data extraction within the user interface by allowing bots to retrieve information from specific elements.
Motivation: Data extraction is one of the most frequent and fundamental requirements in RPA. Many business processes require bots to retrieve information. The Read Element pattern addresses this requirement, providing a generic way to extract precise state or content properties across different underlying application interfaces.
Pattern variants: The Read element pattern has three variants depending on the type of element, as summarised in Table 3.
Applicability:

- The target element must be successfully identified.
- The automation environment allows reading data through DOM interfaces, accessibility frameworks, or screen-scraping services.
- If using visual recognition methods, the target element must be unobstructed and rendered visibly on the computer screen.

Table 3. Variants of Read element pattern

Variant	Action	Object	Method	Context
1	Read	HTML element	HTML DOM parsing	Web environment
2		HTML element, UI element	UI Automation tree parsing	Web environment, Desktop environment
3		HTML element, UI element, Visual element	Visual recognition	Web environment, Desktop environment, Screen environment

Operation: After locating the target element, the bot uses an appropriate automation method to extract the desired property from the element. If more than multiple elements are designate, the bot can declare a tabular or array variable to store the corresponding properties. Once successfully retrieved, the extracted data can be stored in a temporary variable in the bot's memory, or saved into a persistent file.
Example: A bot automates the processing of customer orders from a legacy CRM desktop application. It first uses visual recognition method to locate the text "Customer ID" on screen as the anchor for the actual ID. Then the bot calls OCR as a service to identify and extract the value of ID. The extracted value is then saved in a temporary string variable.
Tool Evaluation: See Table 4.

Table 4. Tool evaluation of Read Element

Tool	Version	Variant	Score	Justification
UiPath	Studio 2025.0.161	Read HTML element	+	Directly supported through "Get Attribute" and "Get Text".
		Read UI element	+	Directly supported through "Get Attribute" and "Get Text".
		Read visual element	+	Directly supported through "Get OCR Text".
Automation Anywhere	Community Edition build 29082	Read HTML element	+	Directly supported through "Recorder: Capture – Get property"
		Read UI element	+	Directly supported through "Recorder: Capture – Get property"
		Read visual element	+	Directly supported through "OCR: Capture area"
Power Automate	2.53.180.25037	Read HTML element	+	Directly supported through "Get details of element on web page"
		Read UI element	+	Directly supported through "Get details of a UI element in window"
		Read visual element	+	Directly supported through "Extract text with OCR"

1.3 Pattern 3: Observe

Description: The Observe pattern enables a bot to monitor a target object for state changes over time. Instead of checking against a predefined expectation, the pattern detects changes in a specified attribute, property, or structural condition of the object. This interaction ensures that the bot remains synchronized with the execution environment, allowing subsequent actions to proceed only after the required state transition occurs. Unlike the Read Element pattern, which performs a one-time extraction of a value, the Observe pattern continuously

monitors the target over time, either through repeated checks or event-driven mechanisms, capturing dynamic UI changes as they occur.

Motivation: In RPA, bots often need to coordinate actions with dynamic interface changes, such as waiting for a button to become enabled, a field to appear, or a value to update. The Observe pattern provides a mechanism to detect these changes over time and trigger subsequent actions only when the required state transition occurs.

Pattern variants: The Observe pattern variants are defined by the type of target object, as summarised in Table 5.

Table 5. Variants of Observe pattern

Variant	Action	Object	Method	Context
1	Observe	HTML element	HTML DOM parsing	Web environment
2		HTML element, UI element	UI Automation tree parsing	Web environment, Desktop environment
3		HTML element, UI element, Visual element	Visual recognition	Web environment, Desktop environment, Screen environment

Applicability: This pattern applies when:

- The target element must be successfully identified.
- The automation environment allows reading data through DOM interfaces, accessibility frameworks, or screen-scraping services.
- If using visual recognition methods, the target element must be unobstructed and rendered visibly on the computer screen.
- Continuous monitoring or event-driven observation is feasible, e.g., the bot can poll or listen for changes without interfering with other UI interactions.

Operation: The Observe pattern begins once the target object is successfully identified, typically through the Find Element or Read Element patterns. The bot then continuously monitors the object to detect state or property changes, ensuring subsequent actions are triggered only when the required transition occurs. This can be done proactively by repeatedly querying the target object's attribute, property, or state at a defined interval (polling), or passively by using an event listener responding to a specific UI change.

Example: At an online ordering system, a bot is implemented to monitor the order statues. When the status becomes "Shipped", an Email notification shall be sent to the relevant customer. The bot uses HTML DOM parsing method to poll the "Order status" element's value in a 24-hour interval.

Tool Evaluation:

2 Modification Patterns

2.1 Pattern 4: Write element

Description: This pattern enables a bot to write, update, or insert data into a target UI element exposed through the application interface. It operationalises content modification within the user interface by allowing bots to change element values, attributes, or states during task execution.
Motivation: Data entry is one of the most frequent and fundamental requirements in RPA. Many business processes require bots to populate forms, submit credentials, update records, or transfer information between applications. These tasks involve inserting or updating values within user interface elements during task execution. The Write Element pattern addresses this recurring automation requirement by providing a generalised mechanism for modifying interface content regardless of how the target application exposes its user interface.
Pattern variants: Write Element pattern has three variants depending on the context, as summarised in Table 6.

Table 6. Variants of Write element pattern

Variant	Action	Object	Method	Context
1	Write	HTML element	DOM manipulation (script injection)	Web environment
2		HTML element, UI element	UI Automation manipulation	Web environment, Desktop environment
3		HTML element, UI element, Visual element	Hardware simulation	Web environment, Desktop environment, Screen environment

Applicability: This pattern applies when:

- a writable interface element exists,
- the element is successfully identified and activated for input,
- at least one input method is feasible.

Operation: After locating the target element, the bot uses an appropriate writing strategy defined according to available interaction layers. The bot may directly assign the desired value through DOM or UI Automation manipulation when structured UI is accessible. If structured interaction is unavailable, the bot can resolve to simulate virtual keystrokes. Either way, when input is completed, the bot removes the focus from the element to register the updated input.
Example: A bot automates entering product data into an ERP system. The bot first extracts the product name from procurement file, then searching for the product name input field in the ERP systems. Once the input field is located, the bot invokes the UIA pattern `ValuePattern.SetValues(“G.Skill Trident Z5 64GB DDR5”)` to programmatically assign the value.
Tool evaluation: See Table 7.

Table 7. Tool evaluation of Write Element

Tool	Version	Variant	Score	Justification
UiPath	Studio 2025.0.161	Write HTML element	+	Directly supported through "Inject Js Script", "Invoke VBScript", "Set Text", and "Type Into".
		Write UI element	+	Directly supported through "Browser: Run JavaScript", "Browser: Call a JavaScript function", and "Recorder: Capture" when setting to "Set Text".
		Write through Hardware simulation	+	Directly supported through "Run JavaScript function on web page" and "Populate text field on web page" while input mode set to "Hardware Events".
Automation Anywhere	Community Edition build 29082	Write HTML element	+	Directly supported through "Set Text" and "Type Into" with input mode set to "Simulate", while both allow explicit overwrite/append control.
		Write UI element	±	Supported through "Recorder: Capture" when "Action to take on object" is set to "Set text".
		Write through Hardware simulation	+	Directly supported through "Populate text field in window" while "Simulate" is enabled. This function supports explicit overwrite/append control.
Power Automate	2.53.180.25037	Write HTML element	+	Directly supported through "Set Text", and "Type Into" when mode set to "Hardware Events".
		Write UI element	+	Directly supported through "Simulate keystrokes".
		Write through Hardware simulation	+	Directly supported through "Send keys" with enable "Send Text as hardware keys"

2.2 Pattern 5: Delete element

Description: This pattern enables a bot to remove a target element from the DOM or UI hierarchy, or a piece of the content it holds, such as the text in a text field. Unlike the Write element pattern, whose purpose is to put data into an element, the Delete element pattern takes something away: the element itself, or the data it holds.
Motivation: In RPA, workflows often require the removal of obsolete, temporary, or incorrect UI elements or data. Delete pattern removes such unwanted elements from interfering with subsequent actions. By manipulating the DOM (at the client side) or UI hierarchy, the Delete pattern enables bots to maintain a more predictable UI for subsequent UI interactions. Workflows also often require clearing data held by an element, such as pre-filled or incorrect values in a text field, or sensitive information that must not be retained, without removing the element itself.
Pattern variants: Deleting an element applies exclusively to structured UI contexts where elements are programmatically identifiable within a hierarchy. It does not apply to image-based automation scenarios, where interface components are treated as visual artefacts rather than manipulable structural objects. Deleting content can also be achieved through hardware simulation by sending virtual delete keystrokes to a focused element. This pattern has three variants, as summarised in Table 8.
Applicability: This pattern applies when:

- The element is programmatically identifiable in the DOM (web) or UI automation tree (desktop), or visually identifiable on the screen.
- The automation environment supports structural manipulation via DOM access, accessibility-level operations, or virtual keystrokes.
- To delete content, the element is editable, such as a text field that is not read-only.
- To delete element, it does **NOT** apply in screen-based/visual environments, where elements can be identified but cannot be structurally deleted.

Table 8. Variants of Delete pattern

Variant	Action	Object	Method	Context
1	Delete	HTML element	HTML DOM manipulation	Web environment
2		HTML element, UI element	UI Automation manipulation	Web environment, Desktop environment
3		Visual element	Hardware simulation	Web environment, Desktop environment, Screen environment

Operation: Once the target UI element is identified, the bot uses the corresponding method:

- To delete an element, the bot removes the element from the DOM (temporarily at the client side) or UI hierarchy, ensuring that subsequent actions operate on a clean and predictable interface.
- To delete content, the bot clears the value of the element through DOM manipulation, or sets the value to empty through UI Automation manipulation. With hardware simulation, the bot first sets the focus on the element, selects the text to be deleted where necessary, and then sends virtual delete keystrokes (e.g., Delete, Backspace). When the deletion is completed, the bot removes focus from the element to register the change.

Example: In a document management system, a bot detects and removes sensitive information, such as personal data. After identifying the target text field, using UI Automation tree parsing. The bot uses UI Automation tree manipulation method (`valuePattern.SetValue("");`) to clear text.
Tool Evaluation: See Table

2.3 Pattern 6: Disable element

Description: This pattern enables a bot to deactivate or suppress a target element without removing it from the underlying UI hierarchy. Rather than performing structural deletion, the Disable element pattern modifies the state of an element so that it no longer interferes with task execution. Typical use cases include suppressing pop-ups, overlays, modal dialogues, or dynamically displayed interface components that block interaction with underlying elements.
Motivation: In RPA, bots frequently interact with applications where UI elements such as pop-ups or overlays interrupt normal task execution. These elements may prevent access to required controls or alter the interaction flow unexpectedly. Removing such elements is sometimes undesirable because they may be required later in the workflow or are controlled by the application logic. The Disable element pattern addresses this challenge by temporarily suppressing or deactivating the element, allowing the bot to proceed with the intended interactions while preserving the underlying UI structure.

Pattern variants: Unlike the Delete pattern, Disable pattern also applies to screen environments, where bots can suppress interfering elements indirectly by simulating user inputs (e.g., clicking the close button of a pop-up). Therefore, this pattern has three variants, as summarised in Table 9.

Table 9. Variants of Disable pattern

Variant	Action	Object	Method	Context
1	Disable	HTML element	HTML DOM manipulation	Web environment
2		HTML element, UI element	UI Automation tree manipulation	Web environment, Desktop environment
3		HTML element, UI element, Visual element	Hardware simulation	Web environment, Desktop environment, Screen environment

Applicability: This pattern applies when:

- A UI element interferes with the intended interaction or prevents access to other required UI elements.
- Structural removal of the interfering element is unavailable or undesirable, and making the element non-interfering is sufficient for the bot to continue its execution.
- The interfering element can be suppressed, deactivated, dismissed, or otherwise placed into a non-interfering state through the available interface.

Operation: The Disable Element pattern applies once the target element is successfully identified. The bot then uses the appropriate method to turn the element into a non-interfering state without removing it from the DOM or UI hierarchy.

In web environment, the bot may modify the element's style properties (e.g., setting `visibility` to `hidden`) to suppress the element. In desktop environment, the bot may use accessibility-level operations to invoke interface controls to turn the element into an inactive state, such as closing a window. Most desktop pop-ups, modal dialogs, and overlays are self-contained windows with OK/Cancel/Close buttons. Therefore, bots can use `InvokePattern.Invoke()` to press a button, which effectively suppresses the interfering element without removing it from the UI hierarchy. In screen environments, the bot first visually identifies the interfering element (e.g., a pop-up or overlay) and then locates its close button or alternative control. Once the target control is identified, the bot sends a virtual mouse click to dismiss the element.

Example: Some websites display an overlay element that blocks all interaction with underlying content until a user acknowledges a pop-up or promotion. When automating a task, such overlays can prevent the bot from clicking buttons or filling forms. The bot identifies the overlay HTML element and

disables it by modifying its CSS properties to disable it, using script such as `overlay.style.visibility = ‘‘hidden’’`.
Tool evaluation: to be done.

3 Control Patterns

3.1 Pattern 7: Open

Description: The Open pattern captures actions that create a new software instance to enable access to a file, application, or web page. This includes operations such as opening a document in its associated application (e.g., opening a `.doc` file in Microsoft Word) or launching a browser to visit a specific URL. Unlike the Activate pattern, which triggers the functional behaviour of an already available element, the Open pattern establishes the required context for subsequent interactions. Launching application executable directly via its directory is considered as a system-level operation, which is outside the scope of UI interaction.
Motivation: UI interactions require explicit designation of their execution context. The Open pattern establishes this context by creating a new software instance for subsequent interactions. It also enables access to specific resources, such as files or web pages, thereby supporting operations like editing or data extraction.
Pattern variants: Depending on the context, Open pattern has three variants, as summarised in Table 10. In practice, different interface elements correspond to distinct resources for the Open pattern. In web environments, HTML elements typically represent hyperlinks or buttons that navigate to a new URL (through a new browser window or tab). In desktop environments, UI elements can include file icons or application shortcuts that open documents or launch application instances. In screen environments, visual elements encompass images representing hyperlinks, buttons, file icons, and application shortcuts, which can be interacted through hardware simulation methods.

Table 10. Variants of Open pattern

Variant	Action	Object	Method	Context
1	Open	HTML element	HTML DOM manipulation	Web environment
2		HTML element, UI element	UI Automation manipulation	Web environment, Desktop environment
3		HTML element, UI element, Visual element	Hardware simulation	Web environment, Desktop environment, Screen environment

Applicability: This pattern applies when:

- A new execution context needs to be established to make a target resource, such as a file, application, or web page, accessible for subsequent UI interactions.
- The target resource can be opened through interaction with a UI element.
- This pattern is **NOT** applicable when the required execution context already exists and only needs to be activated or switched to, or when an application is launched directly through a system-level operation.

Operation: When an opening action is required, the bot establishes a new execution context for the target resource using an appropriate interaction method. This may involve launching a new application instance, opening a new browser window or tab, or loading a resource into its associated application. Depending on the execution environment, the action can be realised through script-based interactions (e.g., DOM methods such as `click()`), structured automation interfaces (e.g., UI Automation patterns such as `InvokePattern.Invoke()`), or hardware simulation. Through this action, the target resource becomes accessible within a newly created context, enabling subsequent interactions.
Example: A bot needs to access a web-based dashboard. It uses `window.open("https://dashboard.exampleURL.com", "_blank");` to create a new browser window and navigate to the URL.
Too evaluation: To be done.

3.2 Pattern 8: Activate

Description: This pattern enables a bot to trigger or invoke the functional behaviour of a user interface element. Activation typically corresponds to executing an action associated with an interactive control, such as clicking a button, selecting a menu item, opening a link, or confirming an operation. Rather than modifying content, this pattern operationalises control execution within the user interface.
Motivation: Activation represents one of the most fundamental interaction objectives in automation, as many application workflows progress through user-triggered controls. Tasks such as submitting forms, navigating interfaces, confirming operations, or initiating system functions require explicit activation of UI elements. This pattern addresses the frequent automation requirement of reproducing user-triggered commands.
Pattern variants: Activate pattern has three variants depending on the context, as summarised in Table 11.
Applicability: This pattern applies when:

- The target element represents an actionable control (e.g., button, hyperlink, menu item, checkbox).
- The element is successfully identified.
- The automation environment allows interaction through exposed interface structures or input simulation mechanisms.

Table 11. Variants of Activate pattern

Variant	Action	Object	Method	Context
1	Activate	HTML element	HTML DOM manipulation	Web environment
2		HTML element, UI element	UI Automation manipulation	Web environment, Desktop environment
3		HTML element, UI element, Visual element	Hardware simulation	Web environment, Desktop environment, Screen environment

Operation: After locating the target element, the bot invokes the activation behaviour using an appropriate interaction method. For content-level activation, the bot triggers DOM events (e.g., `click()` or dispatched mouse events) through script execution. For accessibility-level activation, the bot invokes automation interfaces exposed by the UI Automation tree, such as the `InvokePattern.Invoke()` or equivalent control patterns, allowing direct execution without relying on visual rendering. When exposed interfaces are unavailable, activation can be achieved through hardware simulation, where the bot generates mouse clicks or keyboard events at the element's screen location, emulating physical user interaction.

Example: A bot processes purchase orders in an ERP system. After locating the *"Approve"* button through the UI Automation tree, the bot invokes the element using `InvokePattern.Invoke()`, triggering the approval action. The application then advances to the next processing state automatically.

Tool evaluation: See Table 12.

Table 12. Tool evaluation of Activate

Tool	Version	Variant	Score	Justification
UiPath	Studio 2025.0.161	Activate HTML element	+	Directly supported through "Click" while set input mode to "Chromium API".
		Activate UI element	+	Directly supported through "Click" while set input mode to "Simulate".
		Activate through Hardware simulation	+	Directly supported through "Click" while set input mode to "Hardware events".
Automation Anywhere	Community Edition build 29082	Activate HTML element	+	Directly supported through "Recorder: Capture" when "Action to take on object" is set to "Click", "Left click", "Right click", and "Double click".
		Activate UI element	+	Directly supported through "Recorder: Capture" when "Action to take on object" is set to "Click", "Left click", "Right click", and "Double click".
		Activate through Hardware simulation	+	Directly supported through "Mouse: Click".
Power Automate	2.53.180.25037	Activate HTML element	+	Directly supported through "Click link on webpage", "Click download link on web page".
		Activate UI element	+	Directly supported through "Click UI element in window".
		Activate through Hardware simulation	+	Directly supported through "Send mouse click".

3.3 Pattern 9: Hover

Description: The Hover pattern captures actions where the bot positions the pointer over a target element to trigger context-dependent user interface behaviour without executing the element's primary function. Such interactions typically result in transient UI responses, such as tooltips, pop-ups, previews, or dynamic menus, which provide additional information about the target element. Unlike activation actions, hovering does not alter the state of the application or initiate a new execution context; instead, it facilitates the observation of supplementary interface content.
Motivation: In UI-based automation, bots sometimes need to inspect or retrieve supplementary information that is only exposed through transient interface behaviour. Such information is often not directly accessible from the UI but usually is revealed through a pointer-based interaction, hovering. The Hover pattern captures this design requirement.
Pattern variants: Depending on the types of element, the Hover pattern has two variants, as summarised in Table 13. Although UI Automation patterns can be used to identify elements and observe UI changes triggered by hover, they do not natively support the execution of the hover action itself.

Table 13. Variants of Hover pattern

Variant	Action	Object	Method	Context
1	Hover	HTML element	HTML DOM manipulation	Web environment
2		HTML element, UI element, Visual element	Hardware simulation	Web environment, Desktop environment, Screen environment

Applicability: This pattern applies when:

- The required data or actionable UI elements are not present in the static DOM or visible UI state and cannot be accessed through direct interactions such as click or input.
- The target element provides hover-enabled functionality.
- The bot is capable of interacting with UI elements triggered by the hover action.

Operation: After locating the target UI element, the bot performs the hover action using supported methods, such as dispatching `mouseover` event via JavaScript, or simulate pointer movement through hardware simulation. Then the bot waits for the corresponding UI change to occurs.
Example: In a dynamic dashboard, different categories of orders are represented as proportional square areas, where the size of each square reflects the relative volume of orders. While the visual encoding conveys comparative information,

the exact number of orders is not displayed by default. Instead, this precise value is only revealed when the user hovers over a specific square, triggering a tooltip.
Tool evaluation: to be done.

3.4 Pattern 10: Switch context

Description: The Switch Context pattern enables the bot to change its active execution context between different applications, windows, or containers. This allows the bot to continue interacting with UI elements that reside within another context.
Motivation: In RPA, tasks are often distributed across multiple applications or UI containers. A bot may need to extract data from one application and use it in another. Since UI elements are scoped to their respective contexts, the bot must explicitly switch context to access and interact with elements outside the current scope. Switch context is usually automatically done by operating systems when structured methods (DOM manipulation and UI Automation tree manipulation) are used. This pattern is particularly required when using hardware simulation methods.
Pattern variants: Depending on the method used, the Switch context pattern can be realised in two variants, as summarised in Table 14. Visual representation rendered on the screen is considered as pixels, do not constitute separate contexts and therefore do not require context switching. Transitions between containers on a single web page remain within the same execution context and are managed through direct UI interaction rather than context switching.

Table 14. Variants of Switch context pattern

Variant	Action	Object	Method	Context
1	Switch context	UI element	UI Automation manipulation	Desktop environment
2			Hardware simulation	

Applicability: The Switch Context pattern is applicable when:

- The bot requires to operate on more than one execution contexts.
- The bot can access all execution contexts required.
- This pattern is **NOT** applicable when interactions occur within the same execution context, such as navigating between containers within the same context, or visual regions in the same screen.

Operation: When required, the bot invokes an appropriate method to switch the currently active context to the target context. The target context can be

identified using available identifiers, such as application windows, browser handles, or tab references. Upon completion of the switch, the bot may further verify that the target context is active before proceeding with subsequent actions.
Example: A bot is tasked with transferring customer data from an Excel workbook to a web-based CRM system. The bot first retrieves the required data from the spreadsheet. Then the bot invokes hardware simulation method (by issuing a virtual keyboard shortcut `Alt + Tab`) to switch to the browser, where the designated page is readily loaded. After confirming the browser is the active context, and the URL is correct, the bot proceeds to locate the input field for data input.
Tool evaluation: To be done.

3.5 Pattern 11: Scroll

Description: The Scroll pattern enables the bot to perform scrolling actions to reveal UI elements that are not currently visible within the viewport, allowing subsequent interaction.
Motivation: In many user interfaces, especially web pages and document-based applications, not all elements are visible at once due to limited viewport size. Required elements may be located outside the visible region and must be brought into view before interaction can occur. This is particularly important when using visual recognition methods, where only elements within the visible viewport can be detected and interacted with. In contrast, DOM-based methods may access elements without requiring them to be visible. The Scroll pattern addresses this by enabling the bot to navigate the interface vertically or horizontally to reveal and access such hidden elements.
Pattern variants: Depending on the execution context where the target object is located, the Scroll pattern has three variants, as summarised in Table 15. The Scroll pattern may not be required when DOM-based methods are available, as such methods can access elements without requiring them to be visible. However, scrolling may still be necessary in cases where visibility is required for interaction or when UI behaviour depends on viewport changes (e.g., lazy loading or dynamic rendering).

Table 15. Variants of Scroll pattern

Variant	Action	Object	Method	Context
1	Scroll	HTML element	HTML DOM manipulation	Web environment
2		HTML element, UI element	UI Automation tree manipulation	Web environment, Desktop environment
3		HTML element, UI element, Visual element	Hardware simulation	Web environment, Desktop environment, Screen environment

Applicability: The Scroll pattern is applicable when:

- The target element is currently not visible and cannot be interacted with.
- The target element is located in a scrollable region (e.g., a web page, a document).

Operation: The Scroll pattern is operationalised by adjusting the viewport of a scrollable interface to reveal target elements that are not initially visible. The bot first identifies the scrollable container and determines the direction in which scrolling is required. It then performs scrolling actions using the appropriate method, such as UI Automation–based control of scrollable elements or hardware simulation (e.g., mouse wheel or keyboard input). After each scroll action, the bot checks whether the target element has become visible or accessible. This process is repeated iteratively until the element is revealed or a predefined stopping condition is met. Once the target element is available, the bot proceeds with the subsequent interaction.
Example: A bot is required to click a "Submit" button located at the bottom of a long web page. As the button is not initially visible, the bot performs repeated scrolling actions (via virtual keystroke `PageDown`) until the button appears within the viewport. After detecting the button, the bot proceeds to execute the click action.
Tool evaluation: To be done.

3.6 Pattern 12: Focus

Description: The Focus pattern enables the bot to set the input focus on a target UI element, such as a text field, allowing it to receive subsequent data entry or deletion. The Focus pattern differs from the Activate pattern in that it assigns input focus to a target element without invoking any associated behaviour. In contrast, the Activate pattern triggers the functional behaviour of a user interface element, such as executing a command or initiating an action, thereby potentially modifying the application state.
Motivation: In many user interfaces, input actions such as typing or deleting text require the target element to be in focus. Without explicitly setting focus, input may be directed to an unintended element or fail to execute. The Focus pattern ensures that the correct element is activated and ready to receive input, particularly in scenarios where focus is not automatically assigned.
Pattern variants: Depending on the element types, the Focus pattern has three variants, as summarised in Table 16.
Applicability: The Focus pattern is applicable when:

- The bot requires to focus on the designated element to enable subsequent data entry or deletion.
- The element is not automatically focused and therefore requires a dedicated focus action.

Table 16. Variants of Focus pattern

Variant	Action	Object	Method	Context
1	Focus	HTML element	HTML DOM manipulation	Web environment
2		HTML element, UI element	UI Automation tree manipulation	Web environment, Desktop environment
3		HTML element, UI element, Visual element	Hardware simulation	Web environment, Desktop environment, Screen environment

Operation: The Focus pattern is operationalised by identifying the target element and invoking an appropriate method to assign input focus to it. In web environment, this can be achieved via DOM manipulation method (e.g., `element.focus`). In desktop environment, focus can be directly set using UI Automation tree manipulation (e.g., `SetFocus`). When structured methods are not available, the bot can resolve to hardware simulation (e.g., through mouse clicks and keystrokes `Tab`).
Example: A bot is required to enter a customer name into a text field on a web form. As the field is not automatically focused, the bot first applies the Focus pattern to activate the input field. This is achieved by setting focus on the element using a DOM manipulation method.
Tool evaluation: To be done.

3.7 Pattern 13: Refresh

Description: This pattern enables a bot to update or reload the current state of a user interface element or view to reflect the most recent data. Refreshing typically triggers a re-render or re-query of UI content, ensuring that the displayed information is synchronised with the underlying system or data source.
Motivation: In UI-based automation, data presented on the interface may become outdated due to asynchronous updates or background processing. Bots therefore require a mechanism to re-synchronise the UI state with the latest available data. The Refresh pattern supports reliable automation by ensuring that decisions are made based on up-to-date information, particularly in dynamic or data-intensive environments.
Pattern variants: Depending on the target element that requires refresh, the Refresh pattern has three variants, as summarised in Table 17.
Applicability: This pattern applies when:

- The currently displayed UI state may not reflect the latest state of the underlying system or data source.
- Up-to-date UI state is required before subsequent interaction or decision-making.

Table 17. Variants of Refresh pattern

Variant	Action	Object	Method	Context
1	Refresh	HTML element	HTML DOM manipulation	Web environment
2		HTML element, UI element	UI Automation tree manipulation	Web environment, Desktop environment
3		HTML element, UI element, Visual element	Hardware simulation	Web environment, Desktop environment, Screen environment

– The interface provides a refresh, reload, or update mechanism that can re-synchronise the displayed UI state with the underlying system or data source.

Operation: When required, the bot refreshes the container of the target element using an appropriate method. In web environments, the refresh can be achieved through scripts (e.g., JavaScript execution triggering DOM updates or page reloads). In desktop environments, this can be performed via UI Automation (UIA) patterns, such as invoking control patterns that support update or refresh behaviours. In screen environments, the refresh is realised through hardware input simulation, by sending virtual mouse clicks or keyboard inputs to activate the refresh control.
Example: An RPA bot is tasked with monitoring the status of customer orders in a web-based system. The bot periodically refreshes the page by executing a JavaScript command `location.reload()`. This triggers the browser to resend the HTTP request and re-render the DOM, ensuring the bot can access the most up-to-date information.
Tool evaluation: to be done.